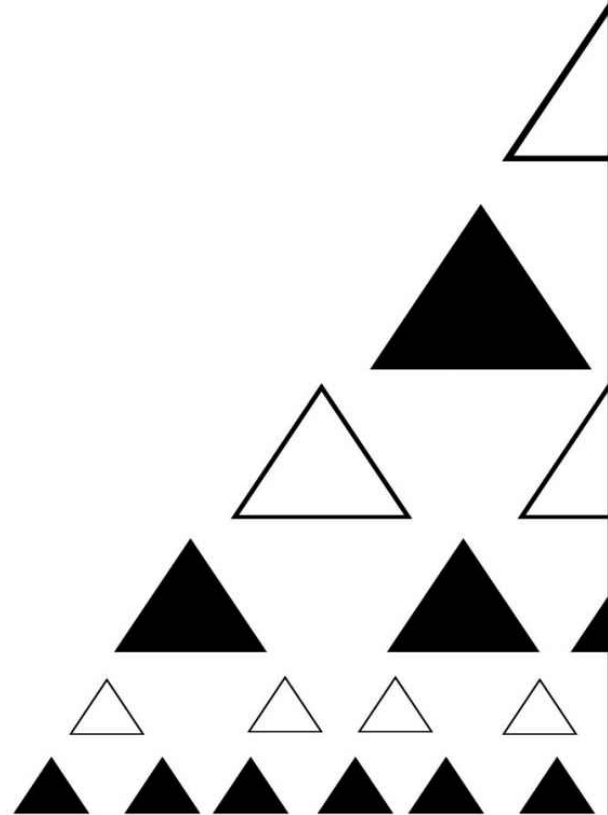


7 Days with **BINARY TREE**

#7DaysOfAlgo

OPENGENUS



7 days with Binary Tree

#7daysOfAlgo

Aditya Chatterjee,
Ue Kiao, PhD

Microsoft Excel uses a Binary Tree to represent the spreadsheet.

On first view, it may look like it uses a 2D array, but it is not the case.

	A	B	C	D	E	F
1						
2						
3	You		Read			
4		Should				
5						
6						
7			This	Book		
8						
9						
10						

Introduction

Binary Tree is one of the most important Data Structure and is equally challenging. With practice and correct way of thinking, you can master it easily and know when to use it in real life problems.

We will attempt one problem every day in this week and analyze the problem deeply.

Our schedule:

- **Day 1:** Introduction to Binary Tree + Problem 1
- **Day 2:** Check if a Binary Tree is Balanced by Height
- **Day 3:** Find nodes which are at a distance k from root in a Binary Tree
- **Day 4:** Modification of Day 3 Problem
- **Day 5:** Find minimum or maximum element in Binary Search Tree
- **Day 6:** Find k^{th} smallest element in Binary Search Tree
- **Day 7:** Modification of Day 6 Problem

On following this routine sincerely, you will get a strong hold on Binary Tree quickly and will be able to attempt interview and real-life problems easily.

#7daysOfAlgo: a 7-day investment to Algorithmic mastery.

With this book, we are solving one problem each day of the week. If you have a month, you should go deeper in Binary Tree as this is a very important topic in Interviews and this book will help you:

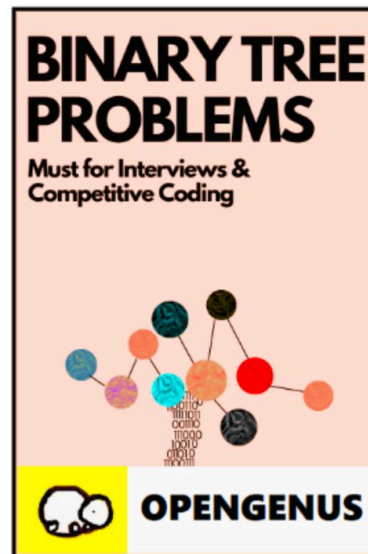
“[Binary Tree Problems: Must for Interviews and Competitive Coding](#)”

by Aditya Chatterjee, Ue Kiao and Srishti Guleria.

(amzn.to/3bZLnc1)

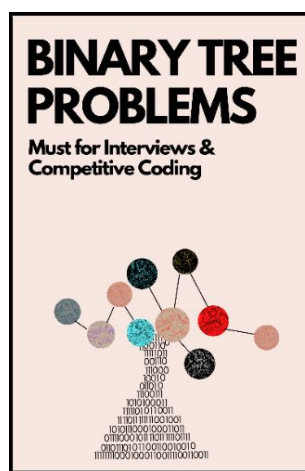
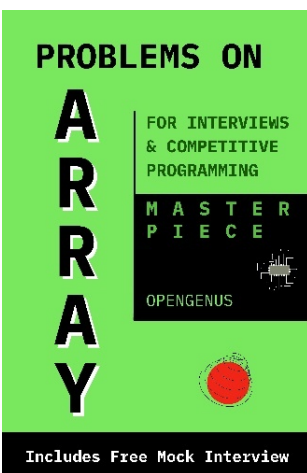
This book has **47** chapters with **373** pages.

Get this
exclusive
book 
now



Other books you must read:

- [Problems on Array: For Interviews and Competitive Programming](#)
- [Binary Tree Problems: Must for Interviews and Competitive Coding](#)
- [Time Complexity Analysis](#)



- [Day before Coding Interview](#) series
- [#7daysOfAlgo](#) series

TABLE OF CONTENTS

#	Chapter
---	---------

Day 1	Introduction to Binary Tree + Problem 1
Day 2	Check if a Binary Tree is Balanced by Height
Day 3	Find nodes which are at a distance k from root in a Binary Tree
Day 4	Modification of Day 3 Problem
Day 5	Find minimum or maximum element in Binary Search Tree
Day 6	Find kth smallest element in Binary Search Tree
Day 7	Modification of Day 6 Problem

Day 1: Introduction + Problem 1

Introduction to Binary Tree

A binary tree is a data structure in which each element links to at most two other elements. It is represented by a starting element from which we can reach the other elements by following the links.

Each element is known as a Node and the starting node of a Binary Tree is known as Root.

In a Binary Tree, every node has at most 2 children that is the left child (left sub-tree) and the right child (right sub-tree). Node of a binary tree will look as follows:



You can see that a node consists of three components:

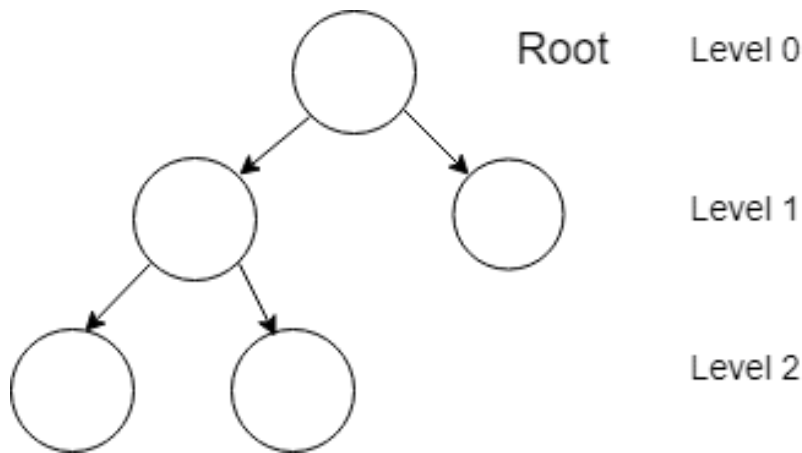
- Pointer to left child node
- Data
- Pointer to right child node

To put this in perspective, think of Linked List data structure where each element links to at most one element. Hence, this is a multi-dimensional version of Linked List.

You can extend this idea to a generalized structure where each node points to at most N other nodes. This structure is known as N-ary Tree. Hence, Binary Tree is an 2-ary Tree.

Suppose there is a binary tree of height "H". The height of root is considered to be 0 as a convention.

Consider this diagram to understand the idea of height of each node:



There are four types of binary trees:

- Complete Binary Tree
- Perfect Binary Tree
- Full Binary Tree
- Balanced Binary Tree

Complete Binary Tree : A Binary Tree is complete Binary Tree if all levels are completely filled except the last level and last level should have all nodes as left as possible.

Perfect Binary Tree : A Perfect Binary Tree is a binary tree in which all interior nodes have two children, and all leaves have the same depth that is same level.

Full Binary Tree : A Full Binary Tree is a tree in which every node other than the leaves has two children.

Balanced Binary Tree : A Balanced Binary Tree is a binary tree in which height of the tree is $O(\log N)$ where N is the number of nodes.

Basic Operations

A Binary Tree supports the following basic operations:

- Insertion: To insert new element into the Binary Tree
- Deletion: To delete a specific element from a Binary Tree

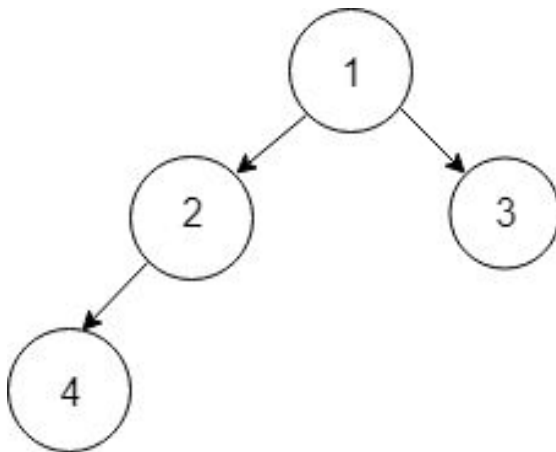
- Search/ Traversal: To go through all elements in a Binary Tree

Insertion operation

Suppose a binary tree is given with few allocated values. As given tree is binary tree, we can insert element wherever we want. Only one property of binary tree should be maintained, and property is that every node can have maximum two children.

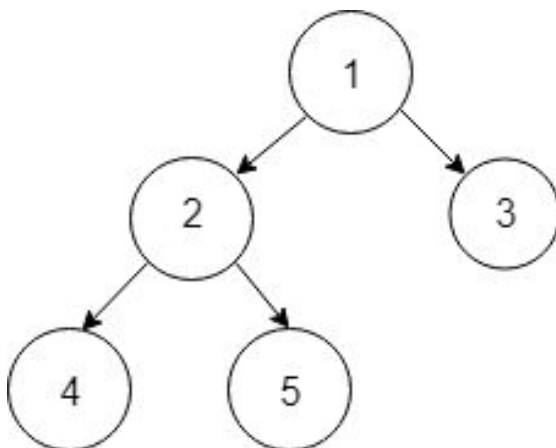
We need to insert the new element wherever we find the node whose left or right child is null.

Given (Original Tree) :



Inserting new element (5) in given tree

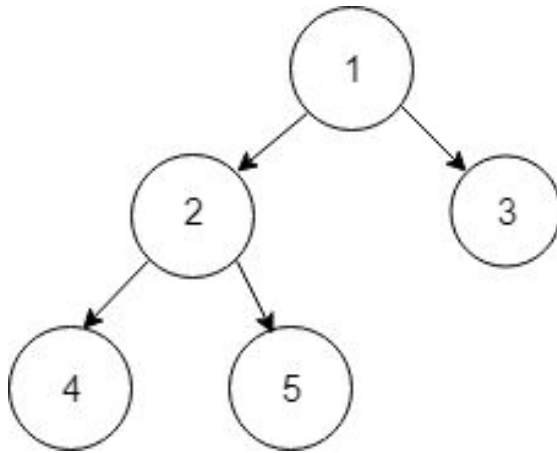
Output (New Tree) :



Deletion operation

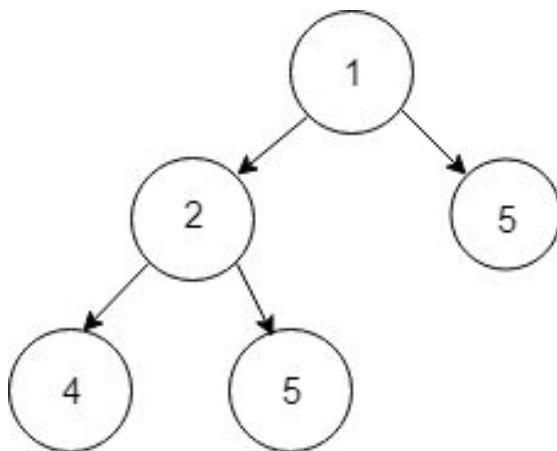
For deletion, first we need to find the element which we want to delete. The basic idea of deletion in a Binary Tree is to replace that element with the right most node (or copy) and then delete the rightmost node.

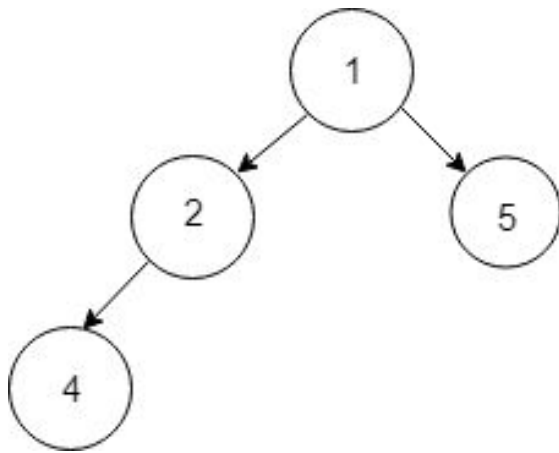
Given (Original Tree) :



Delete data 3.

Deletion steps :





By this way, deletion is done in binary trees.

Time and Space Complexity

Searching : For searching we need to traverse all elements so worst-case time complexity will be $O(N)$ with a space complexity of $O(1)$.

Insertion : Worst case time complexity will be $O(N)$ as it depends on the tree structure. You will understand this well as we move on). The space complexity is $O(1)$.

Deletion : Worst case time complexity will be $O(N)$. The key idea is same as that of Insertion operation which we will understand better in later sections. The space complexity is $O(1)$.

Problem 1: Find height or depth of a Binary Tree

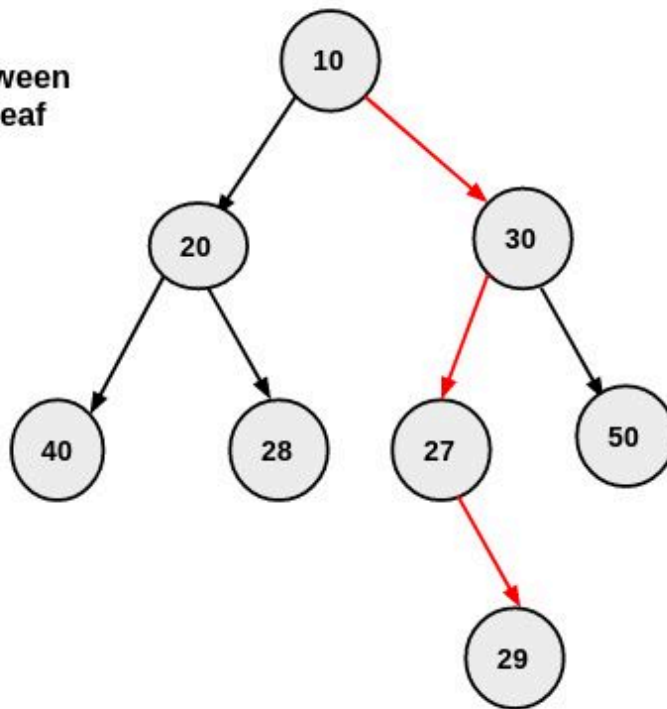
The length of the longest path from the root of a binary tree to a leaf node is the height of the binary tree. It is, also, known as depth of a binary tree.

The height of the root is the height of the tree.

The depth of a node is the length of the path to its root.

We need to find the number of edges between the Binary Tree's root and its furthest leaf to compute the height of tree.

Number of edges between
root and it's furthest leaf
node = **3**.
Hence,
Height of tree = 3.



In above example number of edges between root and furthest leaf is 3.
Hence, height of tree is 3.

We will compute the height of tree by recursively compute the height of left and right subtree and then get the maximum of the two values as the height of tree.

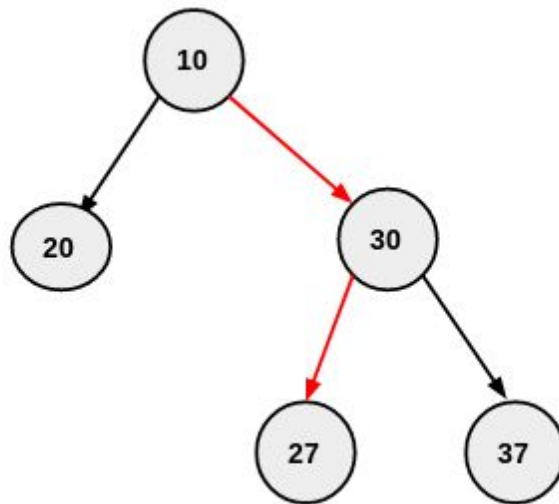
Steps to find height of binary tree

Following are the steps to compute the height of a binary tree:

- If tree is empty, then height of tree is 0.
- If tree is not empty, start from the root
- Find the maximum depth of left sub-tree recursively.
- Find the maximum depth of right sub-tree recursively.
- Maximum depth of the two is (left and right subtree) the height of binary tree.

Example :

Number of edges between
root and it's furthest leaf
node = **2**.
Hence,
Height of tree = **2**.



Start with root node , and recursively find maximum depth of left and right subtree.

Our next node is 20 . 20 is leaf node and as leaf node have no child, height of left subtree is 1.

Now recursively traverse to right subtree, next node is 30. 30 have both left and right node .

First traverse to left side so next node is 27. 27 is leaf node leaf and have no child. left subtree of 30 will return 1.

Next apply same process for right subtree of node 30 . It will return 1.

Height of node 30 is 1 . Number of edges between root to node 30 is 1.

So, total height of right subtree is $1+1=2$.

The height of right subtree is greater than left subtree so height of tree = height of right subtree = 2.

Pseudocode

Following is the pseudocode of the algorithm:

```
int height(Node root)
// return the height of tree
{
    if(root == null)
        return -1;
    else
    {
        int left=height(root.left);
        int right=height(root.right);

        if (left > right)
            return left+1;
        else
            return right+1;
    }
}
```

Complexity

Time complexity : $O(N)$ where N is the number of nodes in the Binary Tree.

It is linear as we are traversing all nodes of Binary Tree recursively and maintaining the height. So, the time complexity is $O(N)$ where N is the number of nodes in the tree.

This can be solved using a standard graph traversal technique like Breadth First Search and Depth First Search as well. Modification of inorder, preorder and postorder traversals make the process of calculating the height of Binary Tree.

Insight:

This is a standard problem but is used as a sub-routine in solve some of the challenging Binary Tree problems. This is a standard definition and approaches to find defined values are important.

Can you imagine how height of a node may be useful?

Day 2: Check if a Binary Tree is Balanced by Height

A Binary tree is said to be balance if:

- The left subtree is balanced for each node.
- Right subtree is also balanced for each node.
- The absolute height difference of left and right subtree is not more than 1.

Further, all empty trees are always height balanced.

There are multiple interpretations of a Binary Tree being balanced and checking for one of these properties lead to checking if the Binary Tree is balanced:

- **Maximum path length of root to leaf node $\leq$ Minimum path length of root to leaf node**

As the Binary Tree is balanced, the maximum and minimum path length from root to a leaf node can be same if the Binary Tree is complete that is the last level “I” has 2^I leaf nodes.

If it is not complete but it is balanced, then a few leaf nodes are missing from the last level and the nodes in the previous level “I – 1” becomes a leaf node. These leaf nodes form the minimum path length. Hence, the maximum difference in path lengths will be 1.

- **Each level “I” has 2^I nodes except the last level where there can be less nodes**

The idea is same as the above point. If all levels are completely filled, then it is a complete Binary Tree and hence, it is balanced. If there are a few nodes missing in the last level, still then it is balanced as the maximum height difference is 1.

- **A complete Binary Tree is always Balanced**

This point should be clear from the explanation of the above two points.

- **All sub-trees of a Balanced Binary Tree are balanced**

This is because as the maximum height difference of the entire tree is 1, the sub-trees cannot have a height difference of greater than 1 as it would add up and make the entire tree unbalanced. Hence, all sub-trees have to be balanced.

We will take a straightforward and efficient approach. The key idea is to check if each sub-tree is balanced or not.

Algorithm:

Step 1: Start

Step 2: If the node is NULL then return 0.

Step 3: If left sub tree is not balanced then return -1.

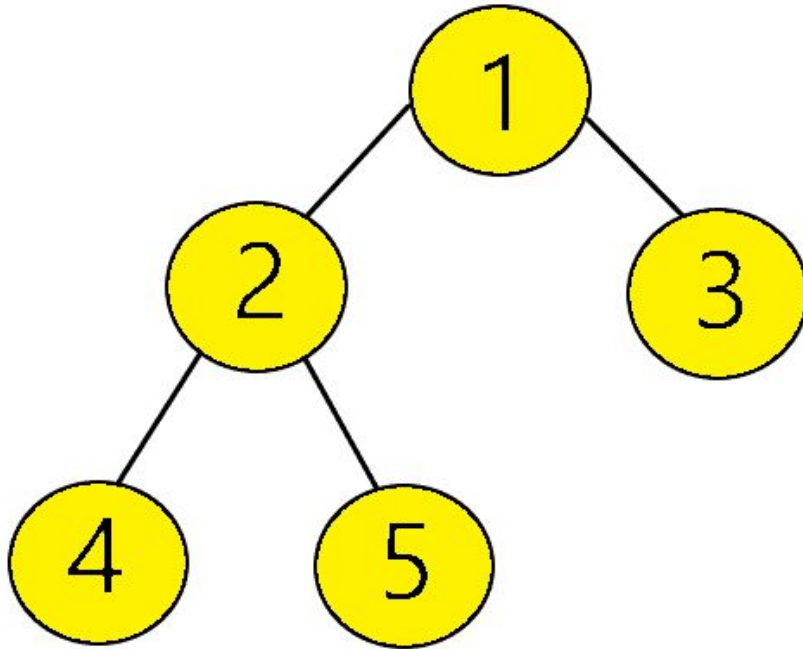
Step 4: If the right sub tree is not balanced then return -1.

Step 5: If the absolute difference between height of left and right subtree is greater than 1, return -1 else return maximum out of left and right subtree's height + 1.

Step 6: End.

Example:

Considering the binary tree below:



Firstly, our node is not NULL.

Hence, we proceed to check the height of left subtree first.

1. For the root node, the height of its left subtree is 2 (from node 1 to node 2 and node 2 to node 4). Similarly, the height of the right subtree is 1 (from node 1 to node 3). So, absolute difference in the height of left and right subtree is 1.

Hence, satisfying the condition that absolute difference of the height of left and right subtree is smaller than or equal to 1.

2. Similarly, for the node 2 the height of its left subtree is 1 (node 2 to node 4) and height of right subtree is also 1 (node 2 to node 5). Hence, the absolute difference of their heights is 0.

Hence, satisfying the condition that absolute difference of the height of left and right subtree is smaller than or equal to 1.

3. For node 4, node 5 and node 3 the absolute difference in their left and right subtrees is 0 since they do not have any children nodes.

Hence, satisfying the condition that absolute difference of the height of left and right subtree is smaller than or equal to 1.

This way all nodes satisfy the required condition. Hence, our binary tree is balanced.

Let us implement this via code :

```
// firstly, pass the root as argument to "balanced" function
int balanced(Node root)
{
    // if the root is NULL return 0
    if(root==NULL)
        return 0;

    //check if the left subtree's height is balanced. If no then return -1
    int left_height = balanced(root->left);
    if(left_height == -1)
        return -1;

    //check if the right subtree's height is balanced. If no then return -1
    int right_height = balanced(root->right);
    if(right_height == -1)
        return -1;

    //if absolute difference of left subtree's height and right subtree's
    height > 1 then return -1
    if(abs(right_height - left_height)>1)
        return -1;

    // return maximum out of left subtree's height and right subtree's
    height + 1
```

```
return max(left_height, right_height) + 1;  
}
```

The time complexity of this approach is $O(N)$.

The space complexity is $O(N)$ due to the stack where recursive calls are stored for all nodes till the leaf nodes are reached.

N is the number of nodes in the Binary Tree.

Insight:

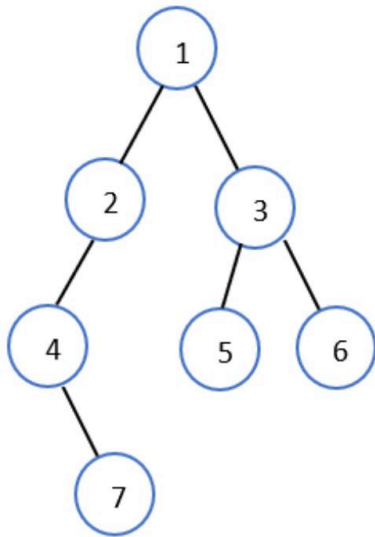
This is an important concept as you will see that a Binary Tree reaches its full potential only when it is balanced by height.

There are techniques to ensure that our Binary Tree is balanced at all times and such trees are known as Self Balancing Binary Tree which we have explored in depth in our Book: “[Binary Tree Problems: Must for Interviews and Competitive Coding](#)”.

Day 3: Find nodes which are at a distance k from root in a Binary Tree

We are given the root of a tree, and an integer K. We need to print all the nodes which are at a distance K from the root. Distance is the number of edges in the path from the source node (Root node in our case) to the destination node.

For example, if the given tree is:



Here, the root is 1. If k is 2, then the output should be 4, 5 and 6 as they are at a distance of 2 from the root.

This problem can be solved using a general traversal technique like:

- Breadth First Search
- Depth First Search
- Level Order Traversal

In Depth First Search (DFS) and Breadth First Search (BFS), we can keep track of the level of each node by adding 1 to each level of traversal. When the level is K, the current node is at a distance of K from the root node.

In Level Order traversal, we shall get all nodes at level K.

This problem can be easily solved using general recursion:

- We will make a recursion function say `printNodes(node root, int k)` .
- This function will recursively call itself in its left and right children, with a distance of $k-1$.
- Finally, when $k=0$ is encountered, we will print the value in the current node. This node will be at a distance of k from the root.

Walkthrough

Let us walk through the procedure with our given example.

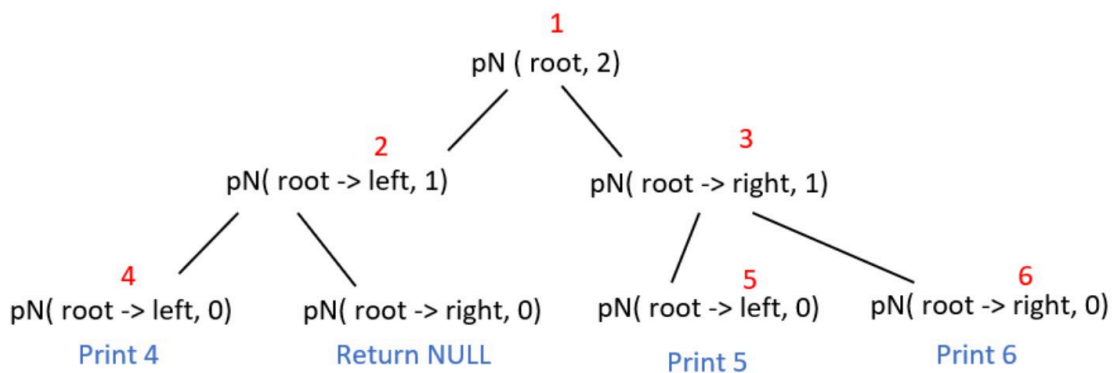
Initially, the root of the tree is 1 and $k = 2$. Since k is not equal to 0, we will recursively call the tree with its left child as the root and $k-1$. Hence, the function `printNodes( root->left, 1 )` will be called.

Our new root will then be 2 and $k = 1$. Again, k is not 0, hence we will call the function `printNodes( root->left, 0 )`.

Now, our root is 4 and this time, $k=0$. So, we will print the data in the given root, as this node will be at a distance of $k=2$ from the original root.

Similarly, the recursive function will be called in the right subtrees until we get a NULL value, or until $k=0$.

The following graph depicts the recursion route. (The function name `printNodes` is abbreviated to `pN`)



Pseudocode:

```

void printNodes(node root , int k)
{
    if(root == NULL)
        return;

    if( k == 0 )
    {
        print( root->data );
        return ;
    }
    else
    {
        printNodes( root->left, k - 1 );
        printNodes( root->right, k - 1 );
    }
}

```

Time complexity: $O(N)$

Space complexity: $O(N)$, where N is the number of nodes.

Insight: This is a pretty easy problem compared to the problems we have already explored. The challenge comes in a modification in the problem statement where we need to find all nodes at a distance K from any given node (can be other than the root node).

Day 4: Finding nodes at distance K from a given node

As an extension to our previous problem, we will explore approaches to find out the nodes at distance K from the given node.

Let us assume we have a binary tree. Let the node given to us be 'Target Node'. Now, we need to find all nodes at distance k from 'Target Node'. Those k nodes could be parent as well as child nodes of the 'Target Node'.

We can follow two major techniques:

1. Breadth First Traversal:

In this method, we perform the Breadth First Search and recursively print the k left child nodes of 'Target Node'. Then in similar fashion print the k right nodes of 'Target Node'. Afterwards, we can traverse the parent node.

2. Percolate Distance:

In this method, we perform the Depth First Search and then at each node, we calculate the distance from the 'Target Node'.

Breadth First Traversal:

1. For the first approach of Breadth First Search we have to convert the binary tree to an Undirected graph. The graph can be made by post-order traversal. We can make adjacency list or adjacent matrix.

2. Now, using Breadth First Search, we will find the k nodes around our 'Target Node'. We will keep storing them in array.

3. We return the array containing all nodes.

Complexity Analysis:

The space complexity will be $O(N)$ as we go to each node twice when we make graph.

The time complexity will be $O(N)$ as Binary Tree is always a sparse graph (think why?)

Pseudocode:

```
//function to calculate distance of nodes from target

vector<int> distanceK(Tree* root, Tree* target, int K) {
    if(root == nullptr || target == nullptr || K < 0){
        return {};
    }

    if(K == 0){
        return {target->val};
    }

    // call function Graph to make graph
    unordered_map<int, vector<int>> graph;
    Graph(root, graph);

    // BFS to find target node
    unordered_set<int> visited;

    queue<pair<int, int>> q;
    q.emplace(make_pair(target->val, 0));

    //vector to store nodes
    vector<int> result;

    while(!q.empty()){
        pair<int, int> curr = q.front();
        q.pop();
        visited.insert(curr.first);

        for(auto &neighbor : graph[curr.first]){
            if(visited.count(neighbor) == 0){
```

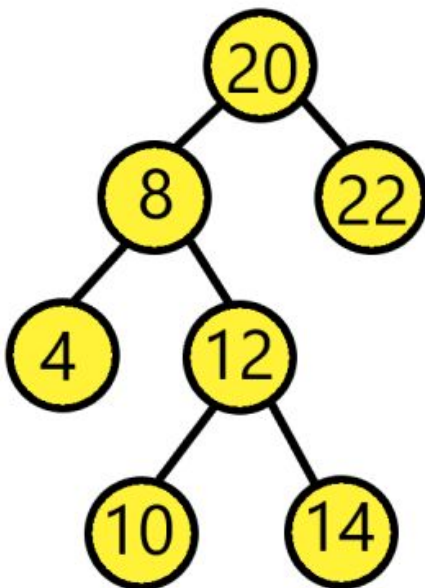
```

        int distance = curr.second + 1;
        if(distance == K){
            result.emplace_back(neighbor);
        }
        else if(distance < K){
            q.emplace(make_pair(neighbor, distance));
        }
    }
}

return result;
}

```

Step by step explanation:



For above tree, firstly we check if the root or target is NULL or if the distance k is less than 0. If so, then we return.

If k is equivalent to 0 then we return the value of the node since it means that we have reached the target node back.

Then, we build a graph by traversing through its left and right subtree and inserting values recursively.

For Breadth First Search, we first create unordered set named "visited" to keep a track of all visited nodes and a queue named "q". The queue will keep pairs of nodes along with its corresponding distance from target.

Then we make vector "result". It will store the final result.

Now, we enter a while loop that traverses until the queue "q" is empty. This means it will keep iterating until all neighboring nodes of target have been visited.

Within While Loop:

- We make a pair "curr", It represents the current pair of nodes and its distance. We store this value from the top value of queue "q" and then pop that value from "q".
- Then, this value is stored in visited vector to keep a record of visited nodes.
- Then, we enter another for loop:
- Then we check if the visited count is 0. If so, we declare variable "distance" and initialize it with second value of "curr" (that is the distance) + 1.
- Now, if the distance becomes equivalent to "k" then we will insert this node to result vector. Else if the distance is less than k, then we must keep iterating. Hence, we insert pair of neighbors and its distance to queue "q".
- In the end after the queue is empty, we get out of our while loop and return result. It will contain all neighboring k nodes of our tree.

The time complexity of this approach is $O(N)$ which is optimal, but the issue is that we need to convert the Binary Tree to an undirected graph which is an overhead.

Percolate Distance:

In this method, we traverse the tree and check if the current node is the 'Target Node'. When we get them, we perform the pre-order traversal to find all neighboring nodes at k distance.

In this method, the striking difference is that we consider the target node as the root node.

Consider an example, suppose we have a root node from which our target node is at distance 3, in right branch. Then, any node at distance $k-3$ in left branch will be returned.

Complexity Analysis:

- The space complexity will be $O(N)$
- The time complexity will be $O(N)$

Implementation:

There are 4 key steps that we take care of :

- If the root==target then return all nodes in the subtree at distance k.
- If the target lies in left subtree of node at distance 'm', then we will find the nodes at distance $k-m$ in right branch.
- In similar fashion we move forward with right branch of node.
- If we cannot find target in either branch of the tree, then we will stop.

Pseudocode:

```
void percolate(root, k)
{
    if(root==NULL || k<0)
        return ;

    if(k==0)
    {
        print( root->data );
        return;
    }
    percolate(root->left, k-1);
    percolate(root->right, k-1);
}
```

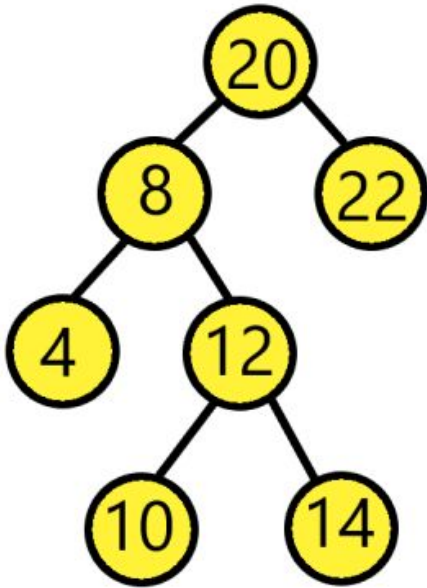
```

int print(root, target, k)
{
    if(root==NULL)
        return -1;
    if(root==target)
    {
        percolate(root,k);
        return 1;
    }
    int left = print(root->left,target,k);
    int right = print(root->right,target,k);

    if(left!=-1)
    {
        if(left==k)
        {
            print( root->data );
        }
        percolate(root->right,k-left-1);
        return left+1;
    }
    if(right!=-1)
    {
        if(right==k){
            print( root->data );
        }
        percolate(root->left,k-right-1);
        return right+1;
    }
    return -1;
}

```

Step by step explanation:



We can follow the below approach to traverse our above tree taken as example and find the k nearest neighbors. Percolate function is a recursive function that traverses left and right sub-tree of our given tree.

Firstly, let us understand the percolate function. If the root is null or k is smaller than 0, we return.

If k is 0 then we return the value of node and return.

In the print function, we have root of tree, target and distance k as arguments. If the root is null, we return -1.

If the root is equivalent to the target, we call percolate function with root and k as arguments.

Then, we declare "left" integer variable and store the value returned by function print in it recursively. This is to check if there is any null node. If there is a null node then value of -1 will be returned to "left".

Similar to "left", we have "right". We follow same procedure for the right subtree too.

Now, if the left is not equal to -1, that is it does not encounter null node, then we check the following:

If the left is equivalent to "k" then we print the value of left i.e root->data.

Since this means that the node is located at distance.

Then we call percolate function again, with distance "k-left-1", because we are going to node that is one step away from the current node. Hence, at left-1 distance. And since we have to traverse k nodes hence k-left-1.

In similar fashion, we check value for "right" integer variable and traverse the nodes. In the end we would have printed all the nodes at k distance in our tree.

Time Complexity: $O(N)$

Space Complexity: $O(N)$ because of stack size.

Insight:

In terms of performance and space, both algorithms are equivalent, but the second approach is important as we are able to avoid modifying the input data (that is the Binary Tree).

In real systems, it is important that the input data is not modified even if there is no such requirements. When needed, a copy of the input is used.

Day 5: Find minimum or maximum element in Binary Search Tree

The problem statement is that we are given a Binary Search Tree and we need to find the minimum or maximum element efficiently.

As we know the Property of Binary search tree. This quite simple

Approach for finding minimum element:

- Traverse the node from root to left recursively until left is NULL.
- The node whose left is NULL is the node with minimum value.

Approach for finding maximum element:

- Traverse the node from root to right recursively until right is NULL.
- The node whose right is NULL is the node with maximum value.

Implementation of the above approaches:

This is the implementation to find the minimum element in a Binary Search Tree:

```
/* Given a non-empty binary search tree,
return the minimum data value found in that
tree. Note that the entire tree does not need
to be searched. */
int minValue(struct node* node)
{
    struct node* current = node;

    /* loop down to find the leftmost leaf */
    while (current->left != NULL)
    {
        current = current->left;
    }
}
```



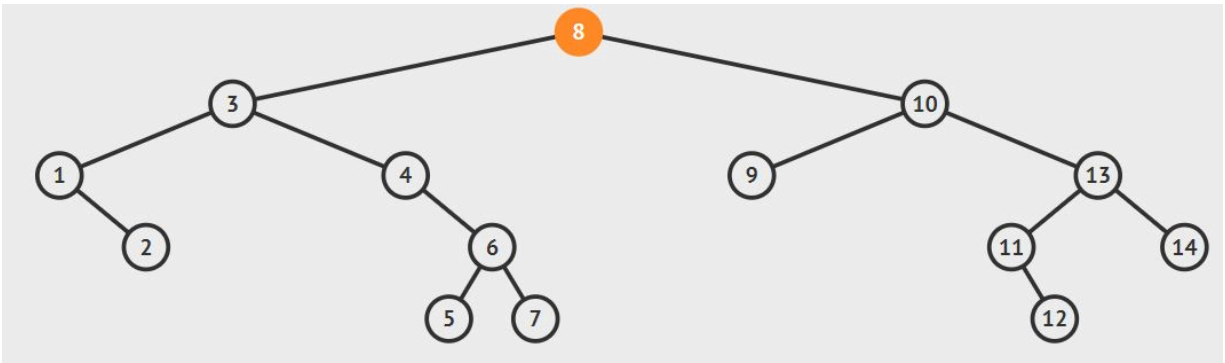
```
    }  
    return(current->key);  
}
```

This is the implementation to find the maximum element in a Binary Search Tree:

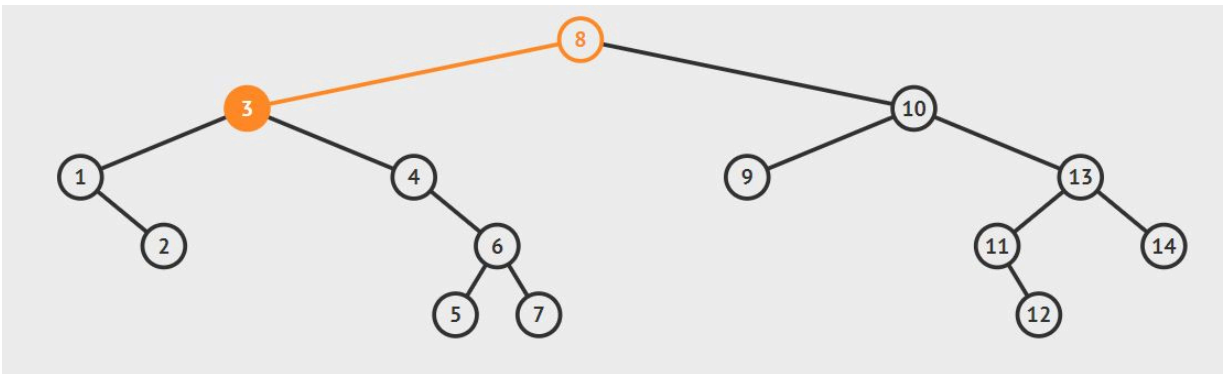
```
/* Given a non-empty binary search tree,  
return the maximum data value found in that  
tree. Note that the entire tree does not need  
to be searched. */  
int maxVal(struct node* node)  
{  
    struct node* current = node;  
  
    /* loop down to find the leftmost leaf */  
    while (current->right != NULL)  
    {  
        current = current->right;  
    }  
    return(current->key);  
}
```

Explanation:

For Finding Minimum value in Binary search tree.
start from root that is 8.

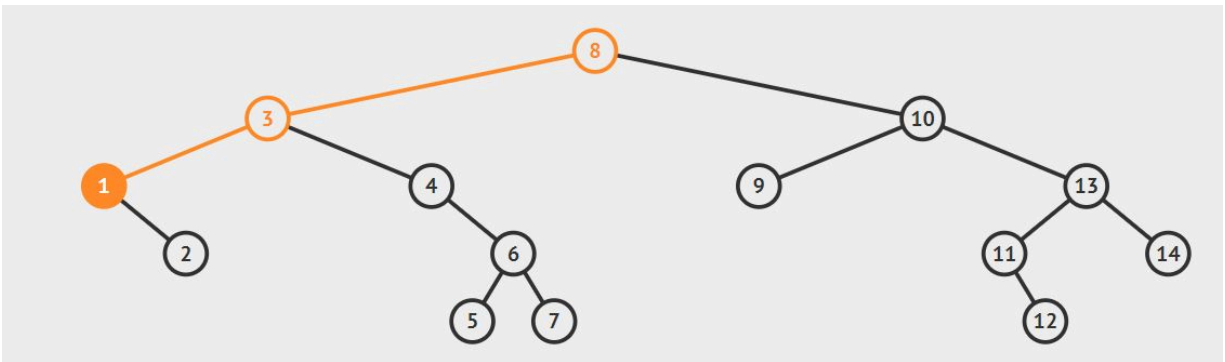


As left of root is not null go to left of root that is 3.



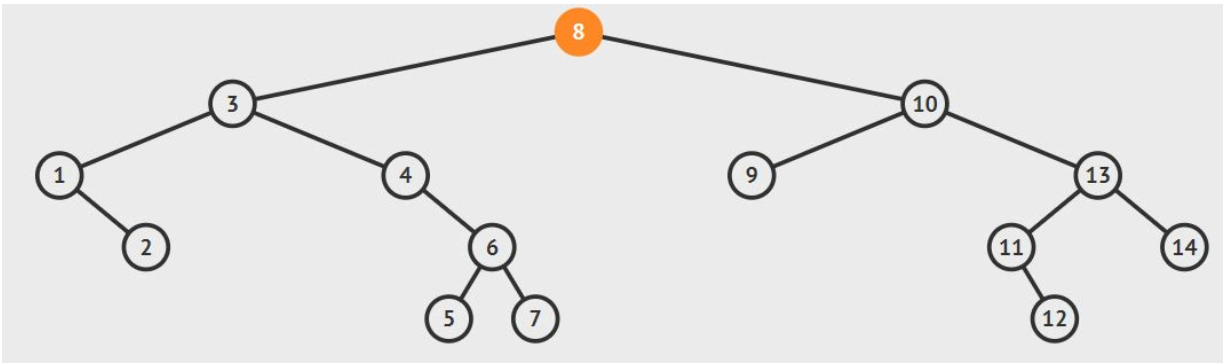
As left of 3 is not null go to left of 3 that is 1.

Now as the left of 1 is null therefore 1 is the minimum element

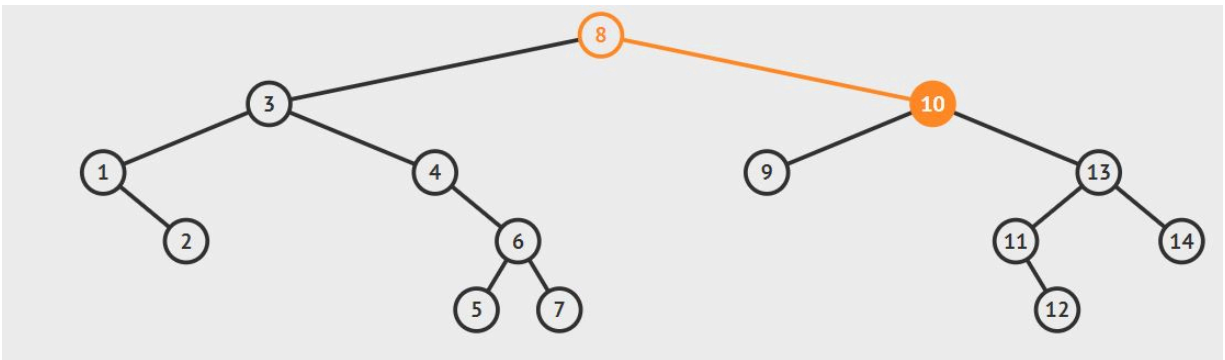


For Finding Maximum value in Binary search tree.

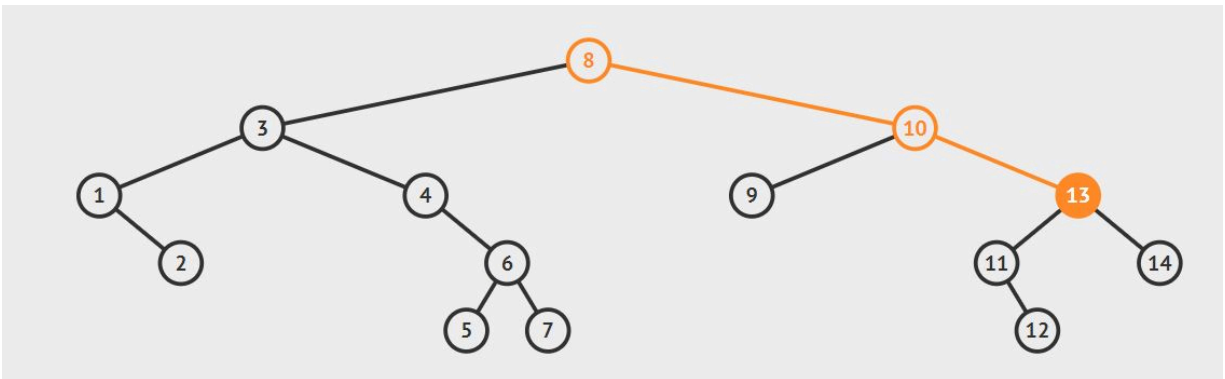
start from root that is 8.



As right of root is not null go to right of root that is 10.

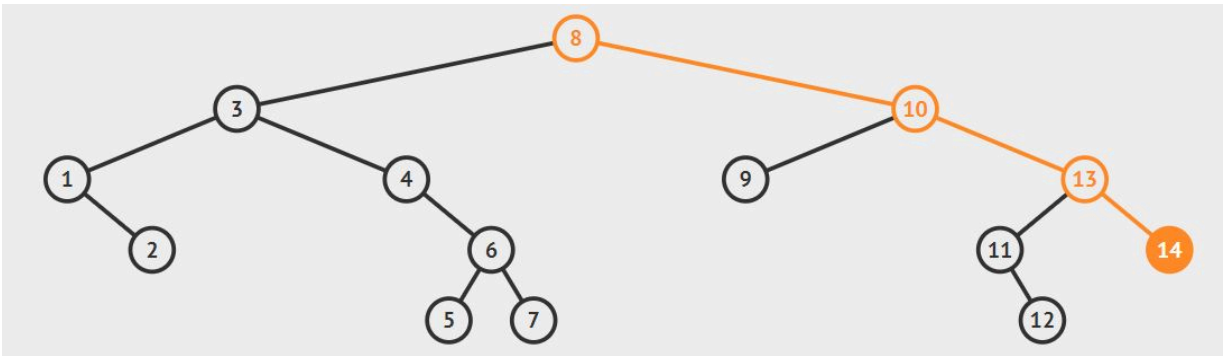


As right of 10 is not null go to right of root that is 13.



As right of 13 is not null go to right of root that is 14.

Now as the right of 14 is null therefore 14 is the maximum element.



Time Complexity:

- $O(N)$ Worst case happens for left skewed trees in finding the minimum value.
- $O(N)$ Worst case happens for right skewed trees in finding the maximum value.
- $O(1)$ Best case happens for left skewed trees in finding the maximum value.
- $O(1)$ Best case happens for right skewed trees in finding the minimum value.
- In the balanced Binary Search tree, the time complexity is $O(\log N)$

N is the number of elements in a Binary Search tree

Space complexity: $O(1)$ as we need not store any information while finding the minimum or maximum element.

Insight:

This problem illustrates the power of Binary Search Tree.

We are able to find the minimum element in $O(\log N)$ time instead of $O(N)$ time because the restriction in the structure of a Binary Search Tree has already done the extra work ($O(N) - O(\log N)$) in other previous operations which we do not observe.

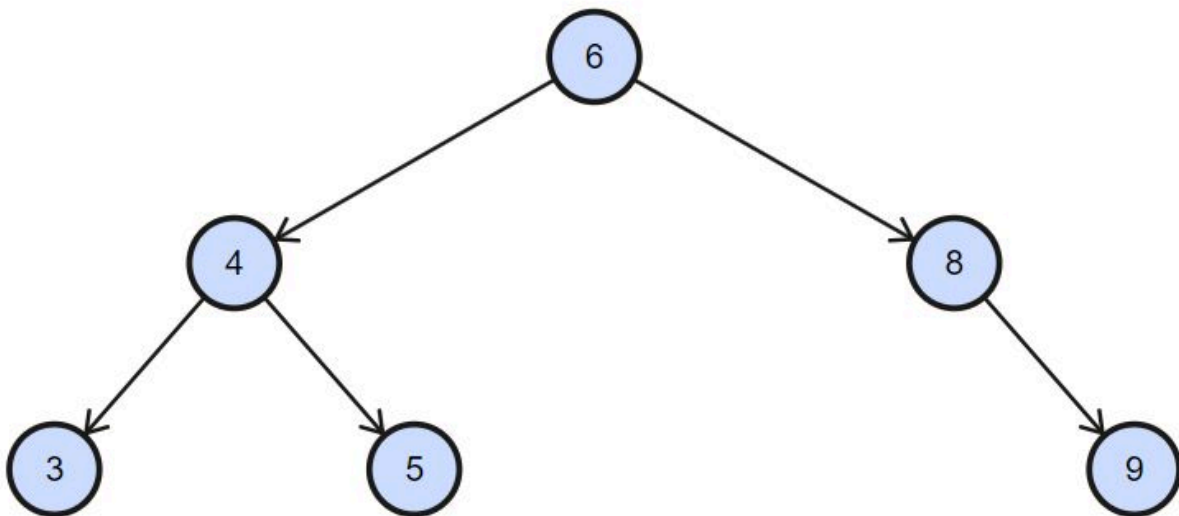
Hence, restrictions in a Data Structure make it suitable for specific problems.

Day 6: Find k^{th} smallest element in Binary Search Tree

In previous problems, we have found the minimum and maximum element in a Binary Search Tree. In this problem, we need to find the k^{th} smallest element in a Binary Search Tree. This does not seem to be simple compared to our previous problem.

Given root of the tree and k as input, output K^{th} smallest element.

For example, in below given tree, for $k=3$ we will get 5.



The idea simple do inorder traversal and store it an array. We know by the property of the binary search tree, inorder traversal gives element of a binary search tree in sorted form.

We will explore two approaches:

- Using inorder traversal ($O(N)$)
- Efficient approach ($O(\log N)$)

Algorithm 1: Using inorder traversal $O(N)$

Do the Inorder traversal of the Binary search tree and store the elements in a vector in sequence.

Then from sorted vector made from above process return the k^{th} element and it is the output we need.

Below is the Implementation of the above algorithm.

```
/* This function traverse the skewed binary tree and
   stores its nodes pointers in vector nodes[] */
void storeBSTNodes(struct node* root, vector<struct node*> &nodes)
{
    // Base case
    if (root==NULL)
        return;

    // Store nodes in Inorder (which is sorted
    // order for BST)
    storeBSTNodes(root->left, nodes);
    nodes.push_back(root);
    storeBSTNodes(root->right, nodes);
}

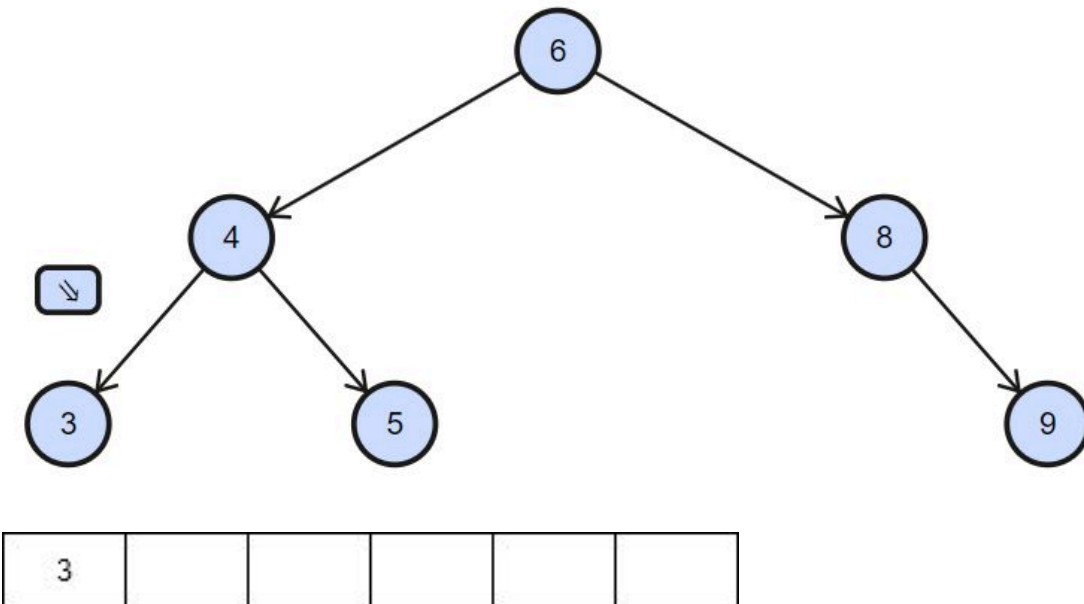
/* Recursive function to construct binary tree */

// This function find Kth Minimum Element in BST
int findKthMinimumElement(struct node* root,int k)
{
    // Store nodes of given BST in sorted order
    vector<struct node*> nodes;
    storeBSTNodes(root, nodes);

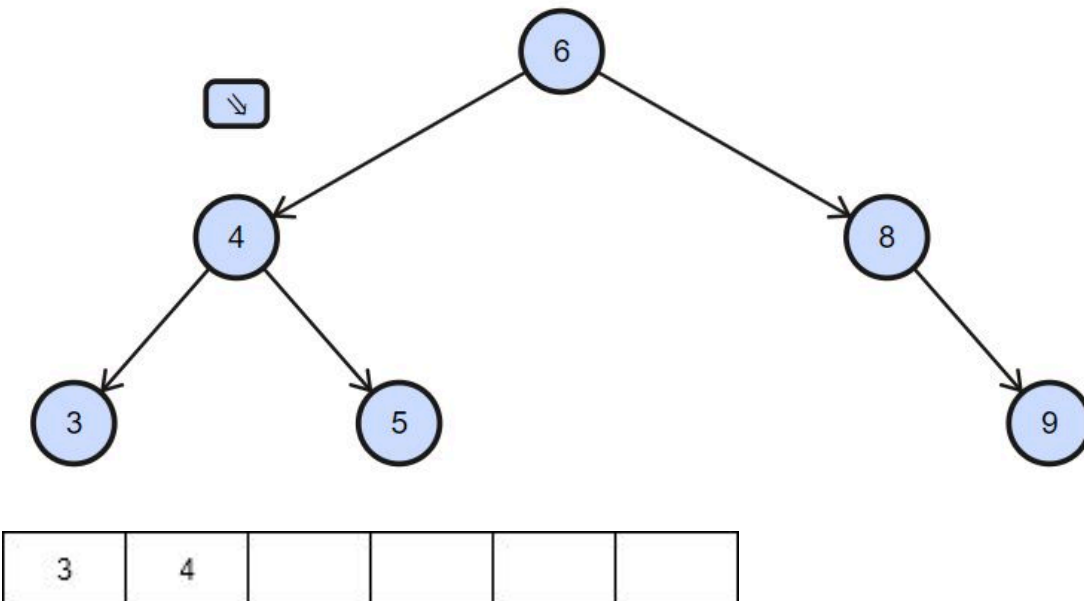
    return nodes[k-1]->key;
}
```

Explanation:

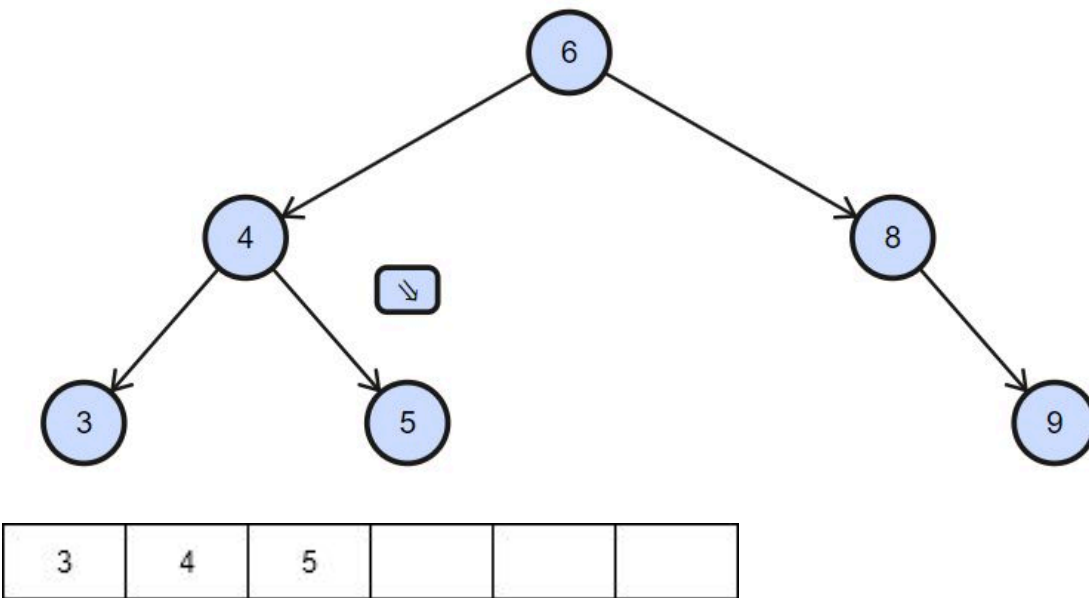
First of all, we go to the left most element of the tree and store it in a vector.



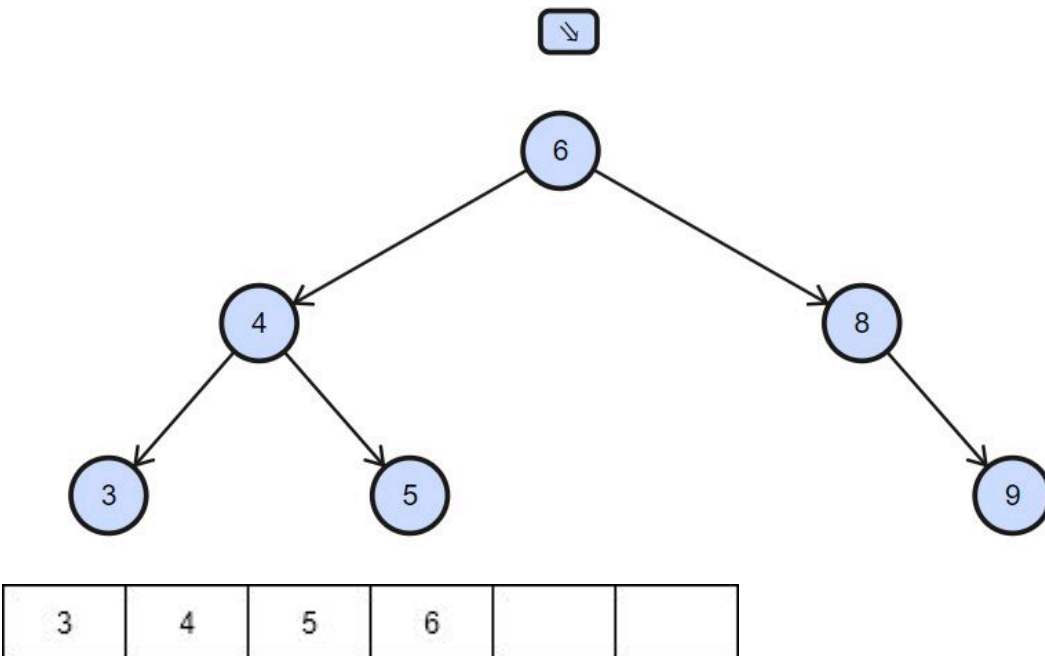
Then return back to parent element and store it in an array.



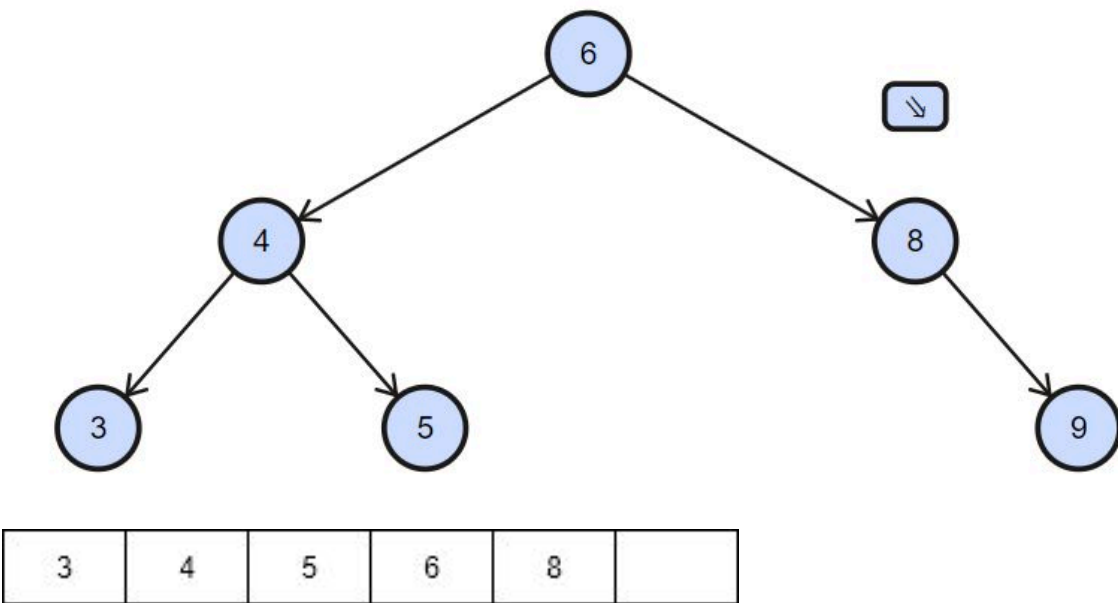
Then go to the right of present node and check if left child is present or right child of the present node then store it in an array and store it in vector.



Then check the right children if it is not existing, then go to parent and then again return to parent of parent because it is visited and store it in vector.

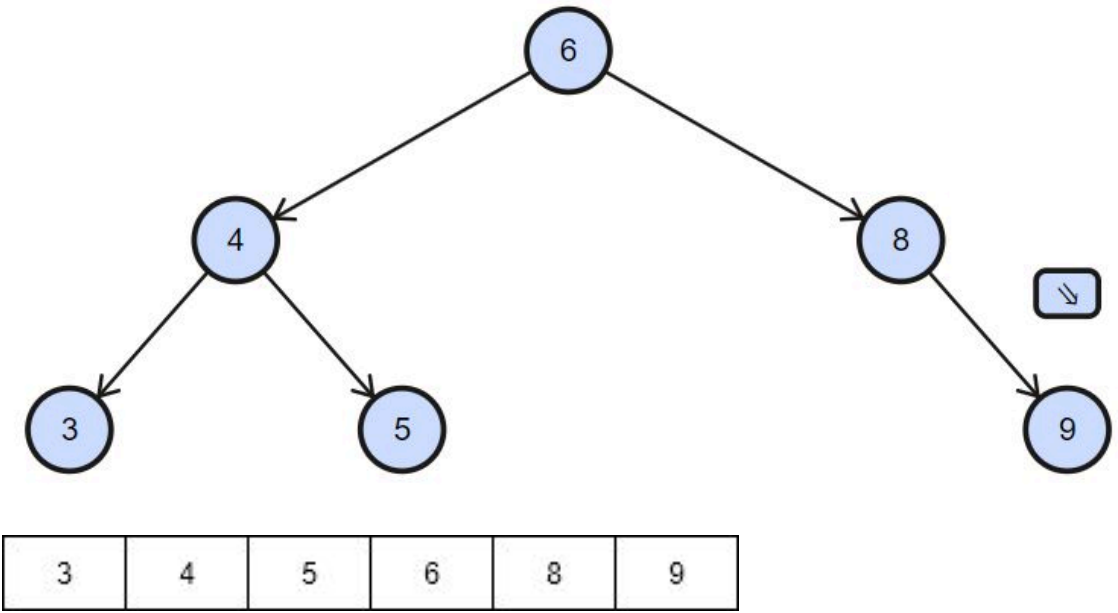


Then go to the right child of present node and check if left child does not exist the store the present node in vector.



The right child of the present node and check if left child exist then go there otherwise store the present element in the vector.

Then check if right child exist then go there if not then, go to the root and now all the nodes are visited therefore return to the main function.



Now required element is the k^{th} one in the array therefore return it and it is required output.

1	2	3	4	5	6
3	4	5	6	8	9



Time Complexity:

Inorder Traversal of Binary search Tree takes $O(N)$ time and fetching the K^{th} element from vector requires $O(1)$ time.

Therefore, all over time complexity of this algorithm is **$O(N)$** .

Efficient Approach $O(\log N)$

The idea to maintain the rank of each node.

We will count the number of nodes in the left subtree of each node so this will give the rank of the node. It will help us to find the element in $O(H)$ time where H is the height of the tree ($H = \log N$ for Balanced Binary Search Tree).

Algorithm:

```

start:
if  $K = \text{root.leftElement} + 1$ 
    root node is the  $K^{\text{th}}$  node.
    goto stop
else if  $K > \text{root.leftElements}$ 
     $K = K - (\text{root.leftElements} + 1)$ 
     $\text{root} = \text{root.right}$ 
    goto start
else
     $\text{root} = \text{root.left}$ 
    goto start

stop:

```

Implementation of above algorithm in C++:

```
int findKthMinimumElement(node *root, int k)
{
    int ret = -1;

    if( root )
    {
        /* A crawling pointer */
        node *pTraverse = root;

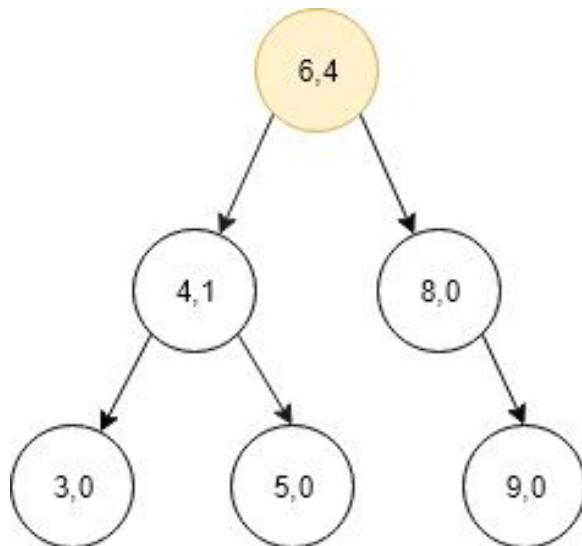
        /* Go to k-th smallest */
        while(pTraverse)
        {
            if( (pTraverse->lCount + 1) == k )
            {
                ret = pTraverse->data;
                break;
            }
            else if( k > pTraverse->lCount )
            {
                /* There are less nodes on left subtree
                Go to right subtree */
                k = k - (pTraverse->lCount + 1);
                pTraverse = pTraverse->right;
            }
            else
            {
                /* The node is on left subtree */
                pTraverse = pTraverse->left;
            }
        }
    }

    return ret;
}
```

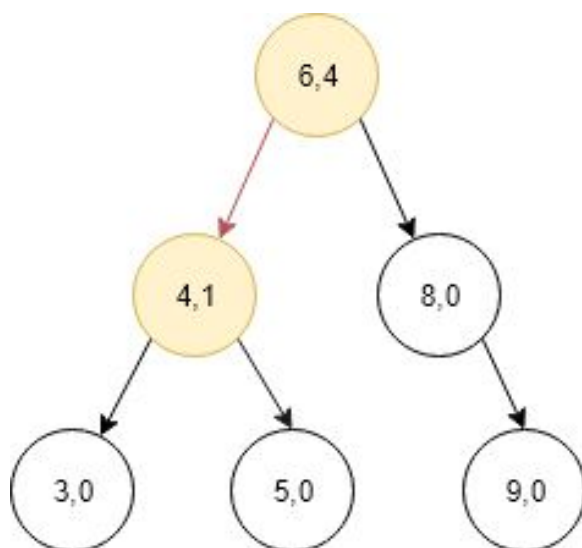
}

Explanation:

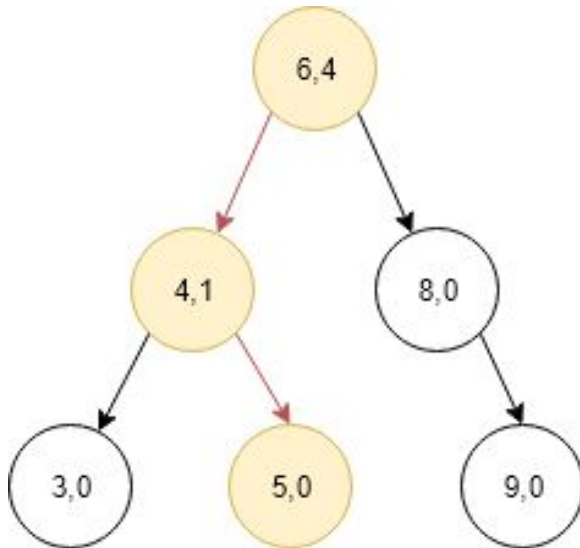
Start with the root compare the lcount with k if k is equal to lcount then break if $k < \text{lcount}$ then go the left otherwise go right. Subtract $\text{lCount} + 1$ from k



In our example, k is smaller then, lcount therefore we come to the left child now again compare lcount of the current node and k. Subtract $\text{lCount} + 1$ from k



As we can see that lcount of the current node is less than k therefore, we go right child now.



Now we can see that lcount of current node is equal to k therefore, return the value of that node which is our output.

Insight:

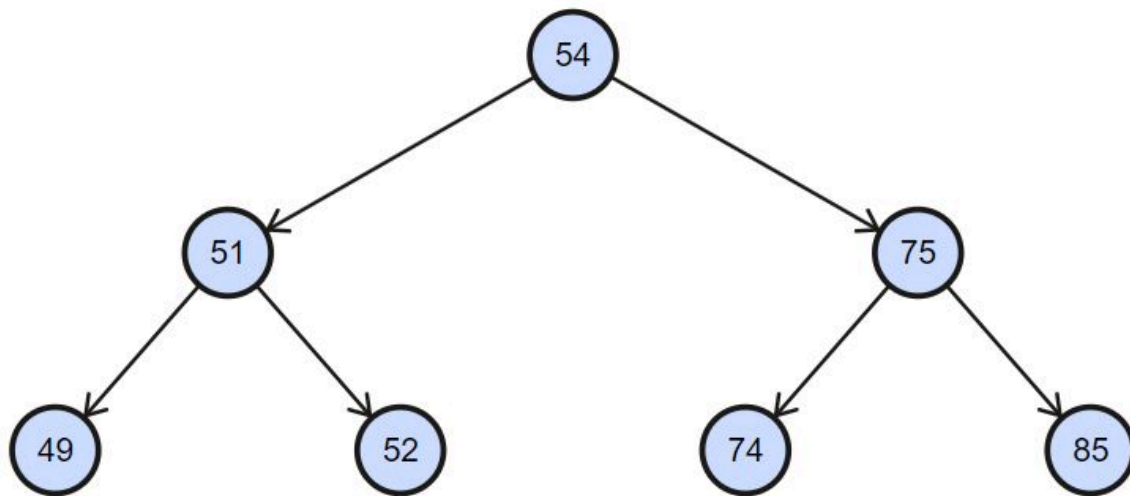
A BST is built to find the smallest or the largest element, but the structure is such that we can find the k^{th} smallest element as well. This is not true in other data structures that we may develop.

Can you find the first k smallest elements? by extending our approach. We will answer in the next problem.

Day 7: Sum of k smallest elements in Binary Search Tree

Given a binary search tree and an integer k , our task is to find that sum of all elements which is less or equal to the k^{th} smallest element in binary search tree.

For example, for given below binary search tree and $k=3$.



Explanation: In above example, k^{th} smallest element is 52 and sum of all the elements which is less or equal to 52 is 152.

We will explore two approaches:

- Naive approach $O(N)$
- Efficient approach $O(\log N)$

Naive Approach

This is a simple approach this takes $O(n)$ time and $O(1)$ extra space.

Algorithm:

- Do inorder traversal of binary search tree and count the visited elements and add all the elements to a variable.

- When count is equal to k return the variable having sum of all the elements until that point and that is the output.

Implementation of above algorithm in C++ is given below:

```
// function return sum of all element smaller than
// and equal to Kth smallest element
int ksmallestElementSumRec(Node *root, int k, int &count)
{
    // Base cases
    if (root == NULL)
        return 0;
    if (count > k)
        return 0;

    // Compute sum of elements in left subtree
    int res = ksmallestElementSumRec(root->left, k, count);
    if (count >= k)
        return res;

    // Add root's data
    res += root->data;

    // Add current Node
    count++;
    if (count == k)
        return res;

    // If count is less than k, return right subtree Nodes
    return res + ksmallestElementSumRec(root->right,
        k, count);
}

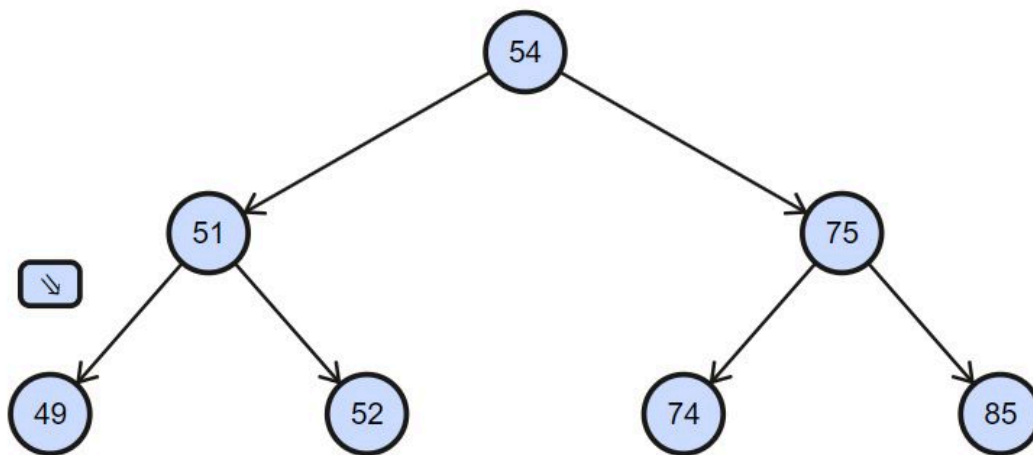
// Wrapper over ksmallestElementSumRec()
int ksmallestElementSum(struct Node *root, int k)
{

```

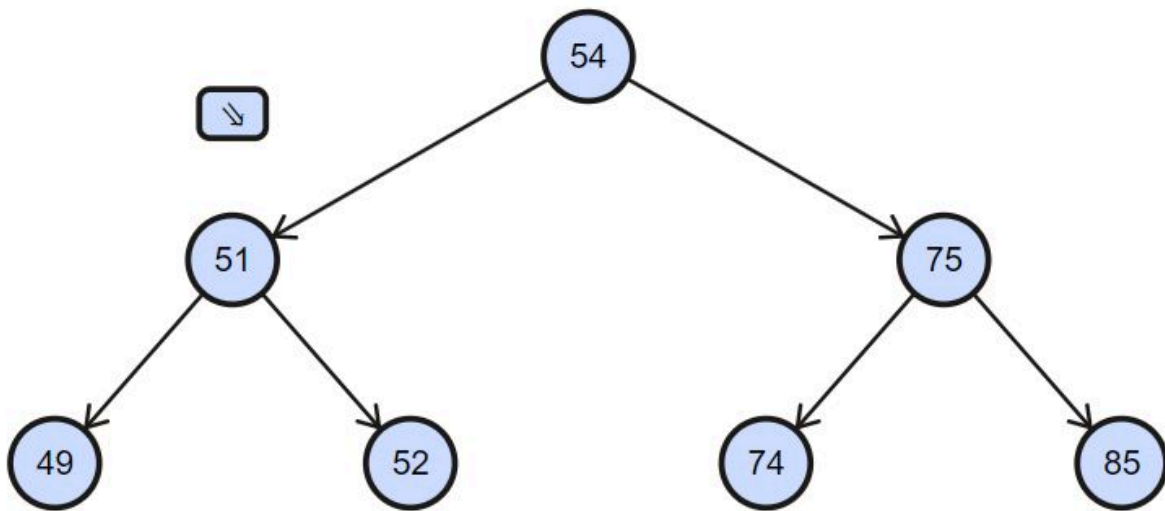
```
int count = 0;  
kSmallestElementSumRec(root, k, count);  
}
```

Explanation:

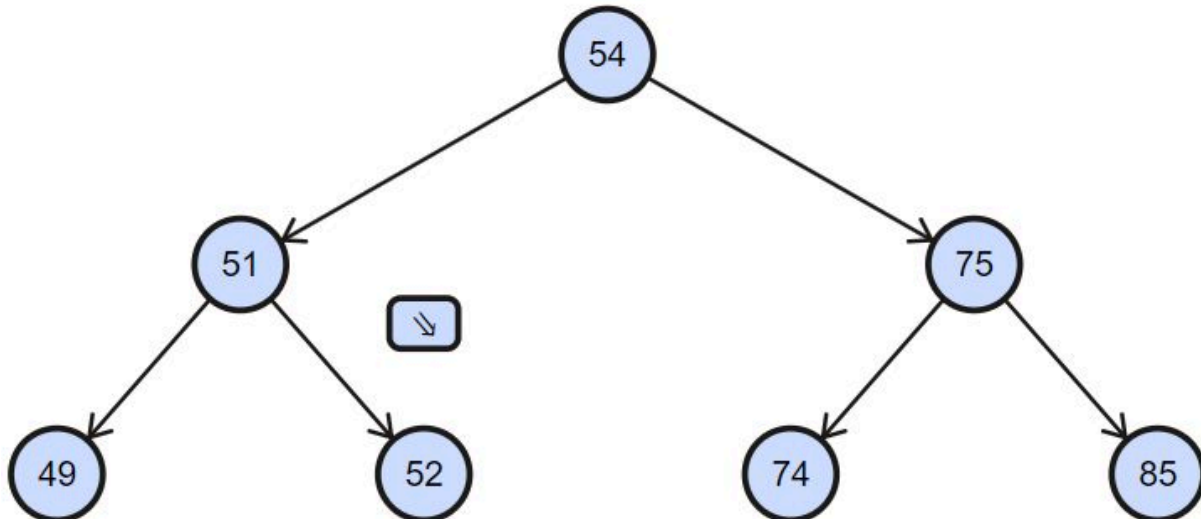
First of all, go to the left most element and add to the variable. In this case, it is res and increment the counter by 1. So, now res = 49 and count = 1. Now, check if count is equal to three. If no, then go another element.



Now current node is 51 add it to the res and increase counter by 1 now res = 100 and count = 2, check if count is not equal to k, then go to the another element.



Now current node is 52 add it to the res and increase counter by 1 now res = 152 and count = 3 check if count is equal to k yes then return res and it is required output.



Efficient Approach

Here we use augmented tree data structure to solve this problem efficiently in $O(h)$ time [h is height of Binary Search Tree].

Structure of Augmented Tree

Binary Search Tree Node contain to extra fields : lCount , Sum

For each Node of Binary Search Tree

- lCount : store how many left child it has
- Sum : store sum of all left child it has

The node is as follows:

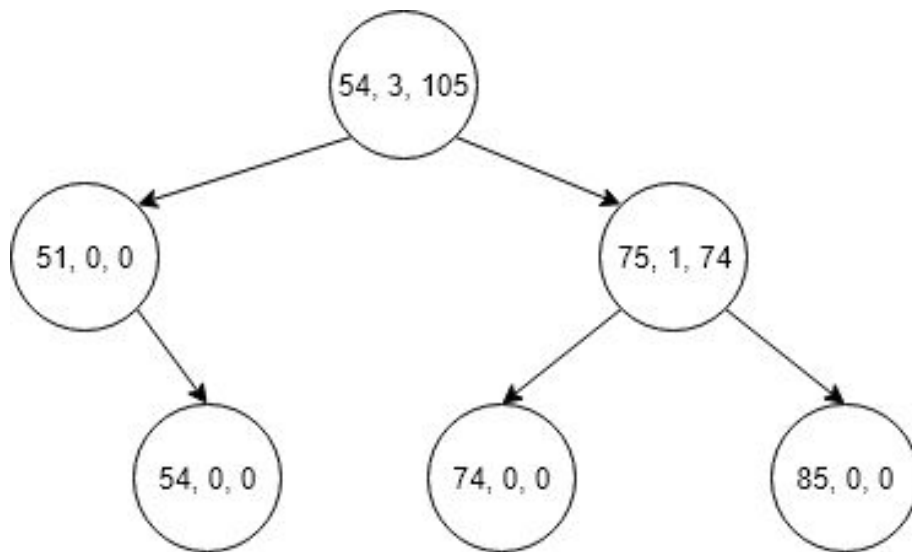
```
/* Binary tree node */  
struct node  
{  
    int data;  
    int lCount;  
  
    node* left;  
    node* right;  
};
```

Insertion in Augmented Tree

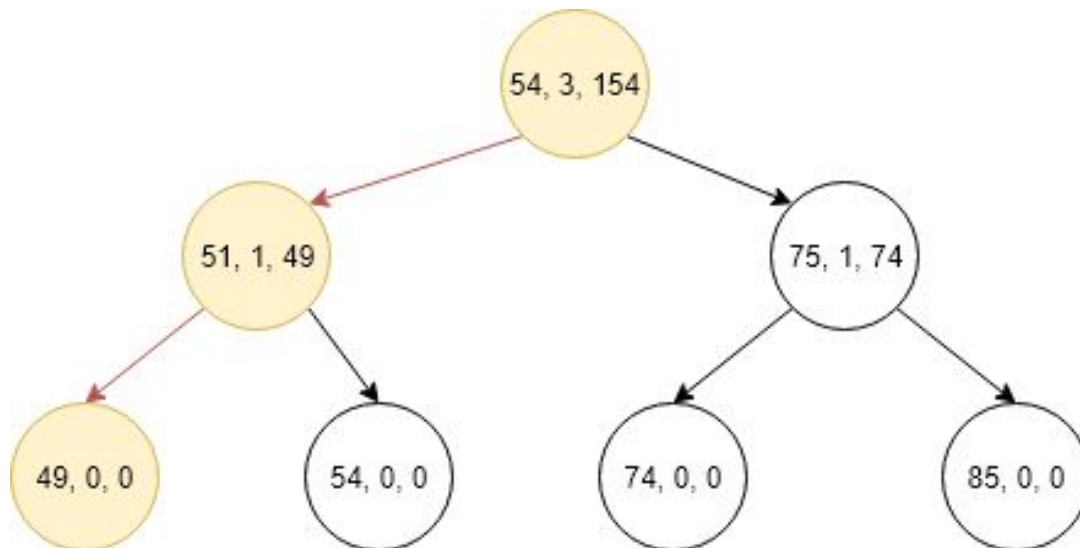
When we insert a node in this tree if key value is less, then current node value then, add key value to the sum of the current node and increment lcount by 1 of that node. if key value is greater than current node then, go to the right similar to binary tree.

Do above process to all the nodes which come in the way till you do not reach the insertion point.

For example: binary search tree given below; we want insert 49 in it.



Now tree becomes as shown below.



Following is the pseudocode to insert an element into our augmented BST:

```

/* Iterative insertion
  Recursion is least preferred unless we gain something
*/
node *insert(node *root, int data)
{
    node *node_t = newNode(data);
    /* A crawling pointer */

```

```

node *pTraverse = root;
node *currentParent = root;

// Traverse till appropriate node
while(pTraverse)
{
    currentParent = pTraverse;

    if( node_t->data < pTraverse->data )
    {
        /* We are branching to left subtree
           increment node count */
        pTraverse->lCount++;
        /* left subtree */
        pTraverse = pTraverse->left;
    }
    else
    {
        /* right subtree */
        pTraverse = pTraverse->right;
    }
}

/* If the tree is empty, make it as root node */
if( !root )
{
    root = node_t;
}
else if( node_t->data < currentParent->data )
{
    /* Insert on left side */
    currentParent->left = node_t;
}
else
{
    /* Insert on right side */
    currentParent->right = node_t;
}

```

```

    }

    return root;
}

```

Algorithm for finding sum of k smallest elements in Binary Search Tree

```

Find Kth smallest element
[ temp_sum store sum of all element less than equal to K ]

ksmallestElementSumRec(root, K, temp_sum)

IF root -> lCount == K + 1
    temp_sum += root->data + root->sum;
    break;
ELSE
    IF k > root->lCount // Goto right sub-tree
        temp_sum += root->data + root-> sum;
        ksmallestElementSumRec(root->right, K-root->lcount+1,
temp_sum)
    ELSE
        // Goto left sun-tree
        ksmallestElementSumRec( root->left, K, temp_sum)

```

Implementation of above algorithm is given below.

```

// function return sum of all element smaller than and equal
// to Kth smallest element
void ksmallestElementSumRec(Node *root, int k , int &temp_sum)
{
    if (root == NULL)
        return ;

    // if we fount k smallest element then break the function

```

```

    if ((root->lCount + 1) == k)
    {
        temp_sum += root->data + root->Sum ;
        return ;
    }

    else if (k > root->lCount)
    {
        // store sum of all element smaller than current root ;
        temp_sum += root->data + root->Sum;

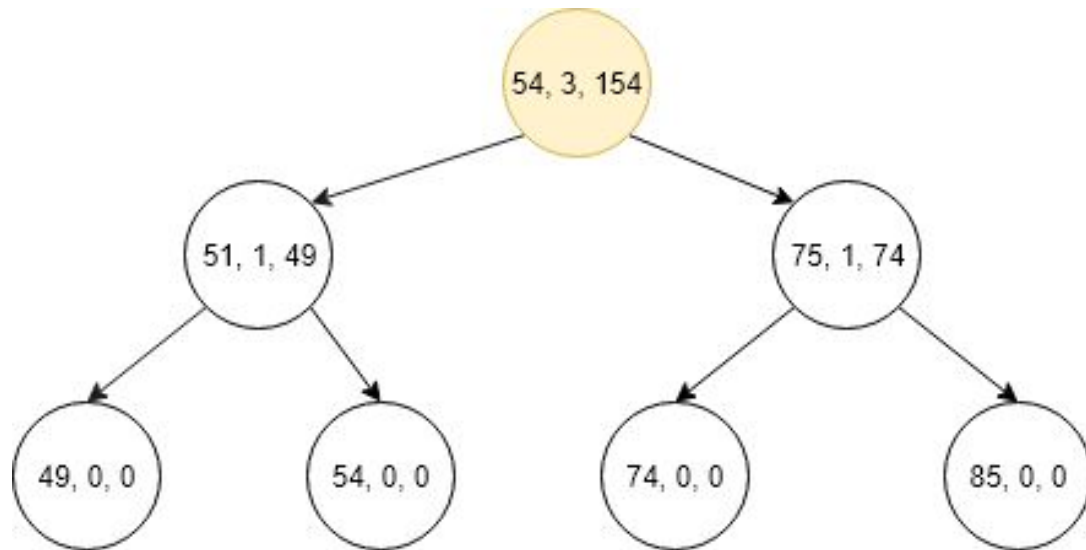
        // decremented k and call right sub-tree
        k = k -( root->lCount + 1);
        ksmallestElementSumRec(root->right , k , temp_sum);
    }
    else // call left sub-tree
        ksmallestElementSumRec(root->left , k , temp_sum );
}

// Wrapper over ksmallestElementSumRec()
int ksmallestElementSum(struct Node *root, int k)
{
    int sum = 0;
    ksmallestElementSumRec(root, k, sum);
    return sum;
}

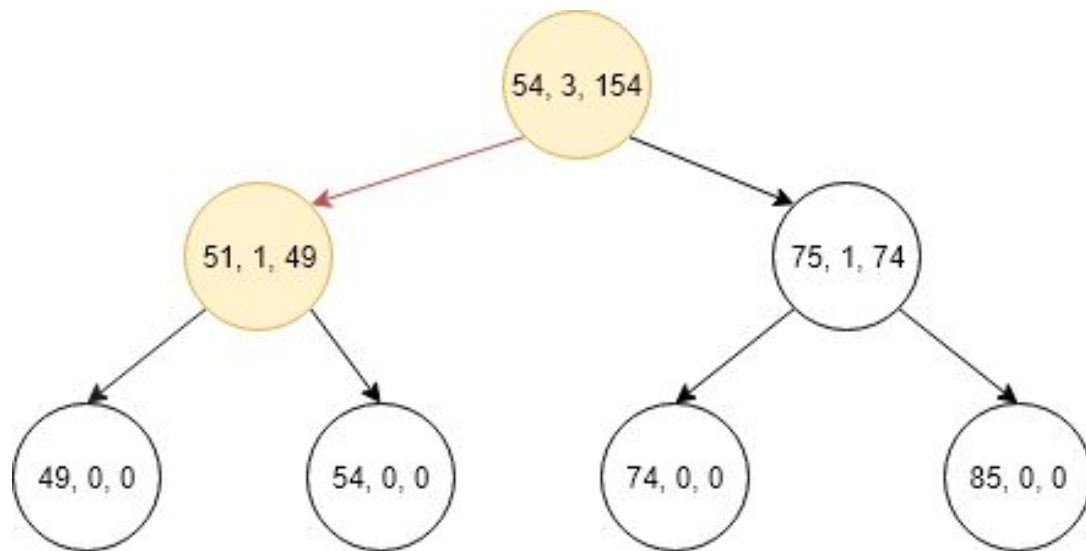
```

Explanation:

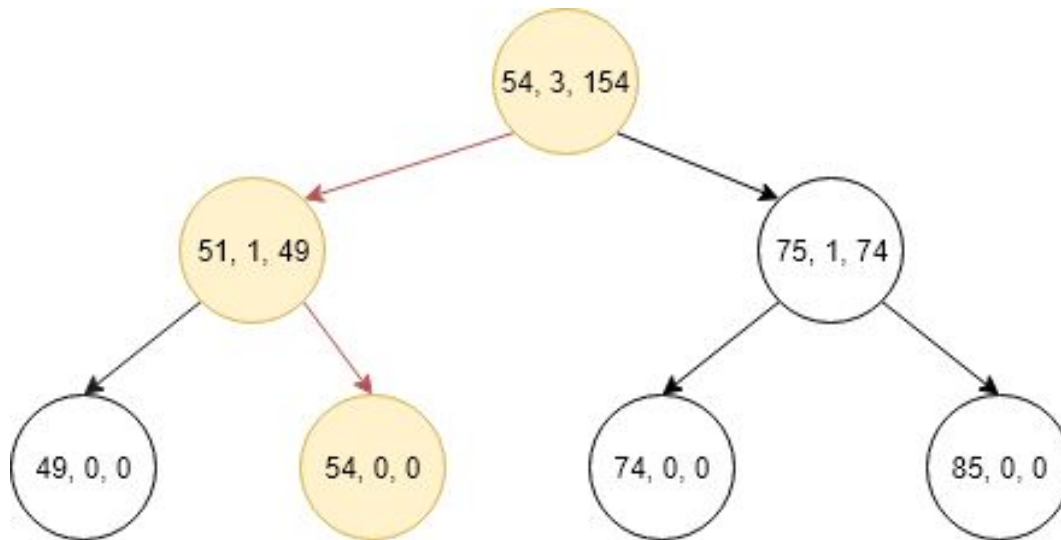
First of all, go to the root and compare k with lcount + 1 if it is equal then temp_sum = 208 and return otherwise if k < lCount then go the left child of the root.



Now compare the lCount + 1 of current node with k if it is equal then temp_sum = 100 and return if k > lCount the go to right and temp_sum = 100 and k = 1.



Now again compare lCount + 1 with k if it is equal then temp_sum = 152 and return.



- Time Complexity: **$O(\log N)$**

If we have our augmented Tree ready, we can find the sum of first K smallest elements in $O(\log N)$ time as in the worst case, we have to traverse to a leaf node.

- Space Complexity: **$O(N)$**

The space complexity is $O(N)$ as we have augmented our Binary Search Tree and store extra attributes for each node.

Insight:

This problem shows that a Data Structure can go beyond its power if it stores extra information. You can imagine that if we store all information, a specific Data Structure can be taken to the edge. This brings in significant space overhead, so Data Structure are tuned according to the problem at hand.

You can imagine this as different species of the same Data Structure.

Conclusion

With the last 7 Binary Tree problems in the last 7 days, you must have a good hold on Binary Tree by now.

Before an Interview, revise these 7 core problems quickly so that you get into the problem-solving mindset and you can solve challenging algorithmic problems easily.

Follow the following steps:

- Define a problem statement and come up with a solution using Binary Tree.
- Analyze the performance compared with other approaches.
- Modify the problem statement slightly and see how the performance of Binary Tree approach is modified.

These bring out the true insights of an algorithmic problem.

Remember:

Binary Tree is a Data Structure that takes in two dimensions of the problem statement in account and enables us to traverse through the data by reducing the potential search candidates by 2.

A unique example is to use Binary Tree to represent an excel sheet. This approach is covered in depth in our book:

“[Binary Tree Problems: Must for Interviews and Competitive Coding](#)”

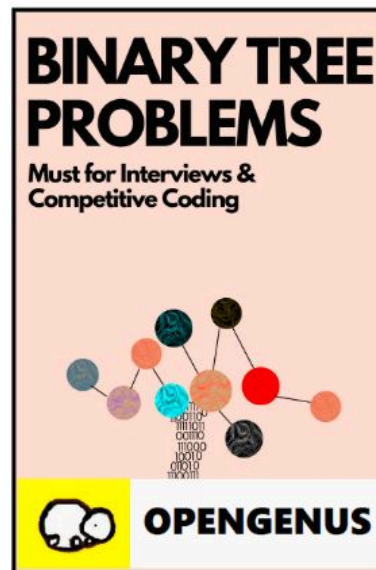
(amzn.to/3bZLnc1)

**Creator of Homebrew fears
BINARY TREE but you need not 😱**

Google: 90% of our engineers use the software you wrote (Homebrew), but you can't invert a binary tree on a whiteboard so fuck off.

7,550 Retweets 469 Quote Tweets 14K Likes

**Get this
exclusive
book 📖
now**





iq.opengenus.org



discuss.opengenus.org



team@opengenus.org



amazon.opengenus.org



linkedin.opengenus.org



github.opengenus.org



twitter.opengenus.org



facebook.opengenus.org



instagram.opengenus.org

Keep solving Computing Problems with us.